

CRIANDO TRABALHOS ACADÊMICOS COM SPECCOMPILER: GUIA PRÁTICO DO TEMPLATE ABNT

SpecCompiler Team

Documento de exemplo do template ABNT.

Orientador: Este documento é auto-demonstrativo

Online

2025

SpecCompiler Team

**CRIANDO TRABALHOS ACADÊMICOS COM SPECCOMPILER:
GUIA PRÁTICO DO TEMPLATE ABNT**

Documento de exemplo do template ABNT.

Orientador: Este documento é auto-demonstrativo

Online

2025

Ficha catalográfica
Elaborada conforme os dados bibliográficos da publicação. Substitua esta página pela ficha catalográfica definitiva fornecida pela biblioteca responsável.
CDD / CDU

FOLHA DE APROVAÇÃO

Documento destinado ao registro da aprovação pela banca examinadora. Substitua esta página pela folha de aprovação definitiva emitida ou validada pela instituição.

Autor(a) _____

Título _____

Data de aprovação _____

Membro da banca examinadora

Membro da banca examinadora

Membro da banca examinadora

*Este documento é dedicado a todos que
buscam uma forma mais simples de criar
trabalhos acadêmicos sem abrir mão da
conformidade com as normas ABNT.*

AGRADECIMENTOS

Agradecemos à comunidade de software livre, especialmente aos desenvolvedores do Pandoc e SQLite, que tornaram possível a criação do SpecCompiler.

Simplicidade é a sofisticação máxima.

— Leonardo da Vinci

RESUMO

Este documento serve como tutorial e demonstração do template Associação Brasileira de Normas Técnicas (ABNT) do SpecCompiler. Através de uma estrutura auto-referencial, apresenta-se como criar trabalhos acadêmicos em formato Markdown que são automaticamente convertidos para documentos DOCX em conformidade com a NBR 14724 ([NBR 14724, 2011](#)). São demonstradas as funcionalidades de elementos pré-textuais, textuais e pós-textuais, além de recursos como tabelas e quadros (com a distinção de formatação exigida pela ABNT), gráficos baseados em dados, figuras, citações bibliográficas e listas automáticas de siglas, figuras, tabelas e quadros.

Palavras-chave: SpecCompiler, ABNT, markdown, trabalhos acadêmicos, NBR 14724.

ABSTRACT

This document serves as a tutorial and demonstration of the SpecCompiler ABNT template. Through a self-referential structure, it presents how to create academic works in Markdown format that are automatically converted to DOCX documents in compliance with NBR 14724. The functionalities of pre-textual, textual and post-textual elements are demonstrated, as well as features such as tables and frames (with the distinct formatting required by ABNT), data-driven charts, figures, bibliographic citations and automatic lists of abbreviations, figures, tables and frames.

LISTA DE FIGURAS

Figura 1 – Capivara descansando - exemplo de figura com legenda.....	16
Figura 2 – Capivara com altura limitada para composição da página.....	18
Figura 3 – Fluxo de criação de documento acadêmico.....	20
Figura 4 – Capivara — exemplo de figura com fonte por citação.....	21

LISTA DE TABELAS

Tabela 1 – Sintaxe do atributo source.....	21
Tabela 2 – Dados regionais — exemplo de tabela CSV.....	23
Tabela 3 – Margens e recuos exigidos pela NBR 14724.....	25
Tabela 4 – Comparação entre AsciiMath e LaTeX.....	26

LISTA DE QUADROS

Quadro 1 – Sintaxes tabulares e a formatação resultante.....	22
Quadro 2 – Elementos pré-textuais conforme NBR 14724.....	23
Quadro 3 – Estilos de formatação em Markdown.....	25

LISTA DE LISTAGENS

Listagem 1 – Comparação de sintaxe: LaTeX vs Markdown.....	16
Listagem 2 – Diferença entre Tabela e Quadro conforme ABNT/IBGE.....	22
Listagem 3 – Principais vantagens do SpecCompiler.....	24
Listagem 4 – Programa Hello World em C.....	25

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
APA	American Psychological Association
IBGE	Instituto Brasileiro de Geografia e Estatística
IEEE	Institute of Electrical and Electronics Engineers
UML	Unified Modeling Language

SUMÁRIO

AGRADECIMENTOS.....	v
RESUMO.....	vii
ABSTRACT.....	viii
LISTA DE FIGURAS.....	ix
LISTA DE TABELAS.....	x
LISTA DE QUADROS.....	xi
LISTA DE LISTAGENS.....	xii
LISTA DE ABREVIATURAS E SIGLAS.....	xiii
1.1 Introdução.....	16
1.1.1 Por que SpecCompiler?.....	16
1.1.2 Extensões ao Markdown.....	17
1.1.3 Estrutura deste Documento.....	17
1.2 Figuras e Ilustrações.....	18
1.2.1 Sintaxe Básica.....	18
1.2.2 Gráficos.....	18
1.2.3 Diagramas PlantUML.....	20
1.2.4 Atributos Comuns para Elementos Visuais.....	20
1.3 Tabelas, Quadros e Listagens.....	21
1.3.1 A Diferença.....	21
1.3.2 Escolhendo o Tipo.....	22
1.3.3 Sintaxe CSV — Tabela.....	23
1.3.4 Sintaxe list-table — Quadro.....	23
1.3.5 Listagens.....	24
1.3.6 Variantes: Invertendo o Padrão.....	25
1.3.7 Equações e Fórmulas com AsciiMath.....	26
1.4 Citações e Referências Bibliográficas.....	27
1.4.1 Tipos de Citação.....	27
1.4.2 Múltiplas Citações.....	27
1.4.3 Configuração: Citeproc e CSL.....	28
1.5 Referências Cruzadas.....	28
1.6 Siglas e Abreviaturas.....	29
1.7 Peculiaridades das Normas ABNT.....	29

1.7.1 Numeração de Páginas.....	30
1.7.2 Capítulos em Páginas Ímpares.....	30
1.7.3 Margens Espelhadas.....	30
1.7.4 Formatação de Tabelas e Quadros (Padrão IBGE).....	30
1.7.5 Template Extensível.....	30
1.8 Conclusão.....	31
1.8.1 Uma Mudança de Paradigma.....	31
1.8.2 Documentação e Relatórios: A Linha que Desaparece.....	32
1.9 Trabalhos Futuros.....	32
1.9.1 Editor Colaborativo em Tempo Real.....	32
1.9.2 Interoperabilidade DOCX (Round-Trip).....	33
REFERENCIAS.....	34
APÊNDICE A – Referência Rápida de Sintaxe.....	35
A.1 Atributos do Documento.....	35
A.2 Figuras.....	35
A.3 Diagramas PlantUML.....	35
A.4 Gráficos.....	35
A.5 Atributos de Dimensões e Fonte.....	35
A.6 Tabelas (dados numéricos).....	35
A.7 Quadros (informação textual).....	36
A.8 Listagens (código e texto preformatado).....	36
A.9 Equações.....	36
A.10 Citações.....	36
A.11 Notas de Rodapé.....	36
A.12 Referências Cruzadas.....	36
A.13 Siglas.....	37
A.14 Inclusão de Arquivos.....	37
A.15 Configuração (project.yaml).....	37
A.16 Seções Pré-textuais Automáticas.....	37
ANEXO A – Normas ABNT Consultadas.....	38

1.1 Introdução

Este documento demonstra, de forma prática, como utilizar o SpecCompiler para criar uma monografia. Cada seção é um exemplo vivo das funcionalidades disponíveis, conforme ilustra a [Figura 1](#).

Figura 1 – Capivara descansando - exemplo de figura com legenda



Fonte: Autor

1.1.1 Por que SpecCompiler?

O ecossistema de ferramentas Markdown para trabalhos acadêmicos—Limarka ([Vieira, 2016](#)), Quarto ([Posit PBC, 2022](#)), RMarkdown—compartilha uma característica comum, todas utilizam LaTeX como backend para geração de PDF. Isso significa que, apesar da simplicidade do Markdown na superfície, o autor ainda precisa de uma distribuição TeX completa instalada, e eventualmente precisará depurar erros de LaTeX quando a abstração “vazar”.

O SpecCompiler adota uma abordagem diferente, não há LaTeX no pipeline. A [Listagem 1](#) ilustra a diferença de sintaxe entre LaTeX e Markdown:

Listagem 1 – Comparação de sintaxe: LaTeX vs Markdown

LATEX	MARKDOWN
<code>\section{Introdução}</code>	<code>## Introdução</code>
<code>\textbf{texto em negrito}</code>	<code>**texto em negrito**</code>
<code>\textit{texto em itálico}</code>	<code>*texto em itálico*</code>
<code>\begin{itemize}</code>	
<code>\item item 1</code>	<code>- item 1</code>
<code>\item item 2</code>	<code>- item 2</code>
<code>\item item 3</code>	<code>- item 3</code>
<code>\end{itemize}</code>	
<code>\begin{figure}[htbp]</code>	<code>```fig:id{caption="Legenda"}</code>
<code>\centering</code>	<code>imagem.jpg</code>
<code>\includegraphics{imagem.jpg}</code>	<code>```</code>
<code>\caption{Legenda}</code>	

<code>\end{figure}</code> <code>\cite{autor2024}</code>	<code>[autor2024] (@cite)</code>
--	----------------------------------

Fonte: Elaboração própria

As principais funcionalidades do SpecCompiler incluem:

1. **Sem dependência LaTeX:** O pipeline utiliza exclusivamente Pandoc e OOXML—nenhuma distribuição TeX necessária
2. **Saída editável e colaborativa:** Diferentemente do PDF (formato de visualização), o DOCX permite controle de alterações, comentários em linha, e revisão por orientadores usando ferramentas familiares (Word, LibreOffice, Google Docs)
3. **Banco de dados:** Estrutura do documento em SQLite permite validação semântica e consultas.
4. **Citações bibliográficas:** Integração com Citeproc para referências no padrão ABNT ([NBR 6023, 2018](#))

1.1.2 Extensões ao Markdown

O Markdown original é deliberadamente minimalista—e essa simplicidade tem um custo. A especificação original não contempla tabelas, fórmulas matemáticas, referências cruzadas numeradas, nem figuras com legendas e fontes. Para escrita acadêmica, essas lacunas são críticas.

O SpecCompiler estende o Markdown com sintaxe adicional para suprir essas necessidades:

- **Tabelas e quadros:** Sintaxes `csv:` e `list-table:`, com a distinção ABNT entre tabela (dados numéricos) e quadro (informação textual)
- **Matemática:** Notação AsciiMath para equações inline e em bloco
- **Figuras numeradas:** Blocos `fig:` com legendas, fontes e numeração automática
- **Referências cruzadas:** Links `[prefixo:identificador] (#)` que se resolvem para “Figura 1”, “Quadro 2”, “Tabela 3”, etc.
- **Listagens:** Blocos `listing:` e `src.<ext>:` para texto e código com moldura ABNT, com contador e lista próprios
- **Gráficos:** Blocos `chart:` alimentados por geradores Lua ou por views SQL do documento
- **Diagramas:** Blocos `puml:` para diagramas UML via PlantUML
- **Inclusão de arquivos:** Blocos `include` para dividir documentos longos em capítulos e apêndices separados
- **Notas de rodapé:** Sintaxe nativa do Pandoc¹ para notas de rodapé numeradas automaticamente

As seções seguintes demonstram essas extensões em uso prático.

1.1.3 Estrutura deste Documento

Este tutorial segue a estrutura padrão de uma monografia conforme NBR 14724 ([NBR 14724, 2011](#)):

¹ Notas de rodapé são suportadas nativamente via Pandoc `commonmark_x`. Esta é uma nota de exemplo que aparecerá no rodapé da página.

- **Elementos pré-textuais:** Capa, folha de rosto, resumo, listas, sumário
- **Elementos textuais:** Introdução, desenvolvimento, conclusão
- **Elementos pós-textuais:** Referências, apêndices, anexos

1.2 Figuras e Ilustrações

As figuras são inseridas utilizando blocos de código com o prefixo `fig:` seguido de um identificador único. Os atributos `caption` e `source` definem a legenda e a fonte da ilustração.

1.2.1 Sintaxe Básica

Para inserir uma figura:

```
```fig:identificador{caption="Legenda da figura" source="Fonte"}
caminho/para/imagem.jpg
```
```

A legenda é posicionada abaixo da figura, seguida da fonte — conforme exige a ABNT.

Quando a imagem original é alta demais para a página, limite a altura diretamente nos atributos do bloco. A largura será ajustada proporcionalmente pelo processador DOCX:

```
```fig:identificador-limitado{caption="Legenda da figura limitada" source="Fonte"
height="5cm"}
caminho/para/imagem.jpg
```
```

A **Figura 2** usa a mesma imagem do exemplo inicial, mas com altura máxima definida para deixar mais espaço para texto, legenda e fonte na página.

Figura 2 – Capivara com altura limitada para composição da página



Fonte: Autor

1.2.2 Gráficos

Gráficos são renderizados usando ECharts², uma biblioteca de visualização de dados. O corpo do bloco contém a configuração JSON do gráfico, e os dados podem vir de duas fontes:

² Apache ECharts — biblioteca open-source para visualização interativa. Documentação em <https://echarts.apache.org>.

- **generator** (ou **view**): invoca um módulo Lua que calcula os dados programaticamente
- **query**: consulta uma view SQL sobre o banco de dados do próprio documento

O capítulo *Referências Cruzadas* demonstra a segunda forma, com um gráfico alimentado pelas relações extraídas deste documento.

1.2.2.1 Geradores Lua para Funções Matemáticas

Geradores Lua são particularmente úteis para visualizar funções matemáticas. Cada gerador implementa uma função específica e pode ser invocado via atributo `generator` ou `view` no bloco de código do gráfico.

Como criar um novo gerador:

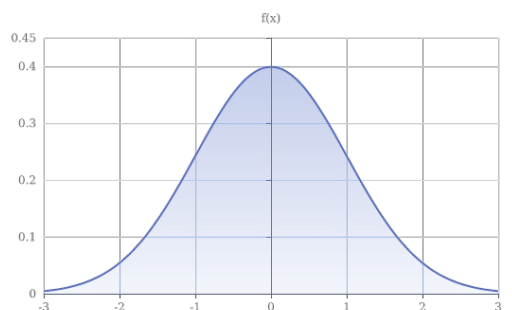
Um gerador é um módulo Lua em `types/views/` que segue este padrão:

```
return {
  kind = "view",
  schema = {
    id = "ID_UNICO",                -- Identificador em maiúsculas
    long_name = "Nome Descritivo",
    description = "O que faz",
    inline_prefix = "prefixo",      -- Sintaxe inline: `prefixo:
param=valor`
    aliases = { "alias1", "alias2" }, -- Nomes alternativos
  },
  hooks = {
    render = function(ctx)          -- Para uso inline
      -- Renderiza resumo de parâmetros
    end,
    dataset = function(dctx)        -- Para dados do gráfico
      local params = dctx.subject.params or {}
      -- Processar params, calcular função, retornar dados
      return { source = {{ "x", "y" }, {x1, y1}, ... } }
    end,
  },
}
```

Exemplo: Distribuição Normal

A [Gráfico 1](#) renderiza a distribuição normal a partir de parâmetros:

Gráfico 1 – Distribuição Normal — gerada via Lua

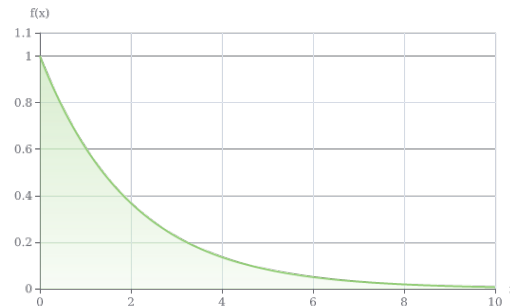


Fonte: O autor

Exemplo: Decaimento Exponencial

A [Gráfico 2](#) implementa uma nova função de decaimento exponencial ($f(x) = e^{(-\lambda x)} + \text{offset}$):

Gráfico 2 – Decaimento Exponencial — $f(x) = e^{(-0.5x)}$



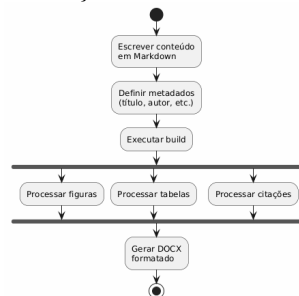
Fonte: O autor

1.2.3 Diagramas PlantUML

Diagramas Unified Modeling Language (UML) podem ser gerados a partir de código PlantUML. O sistema processa os diagramas em paralelo para melhor desempenho.

A [Figura 3](#) ilustra o fluxo básico de criação de um documento:

Figura 3 – Fluxo de criação de documento acadêmico



Fonte: O autor

1.2.4 Atributos Comuns para Elementos Visuais

Os elementos `fig:`, `puml:` e `chart:` suportam atributos adicionais para controle de dimensões e fonte.

1.2.4.1 Dimensões (width e height)

Para controlar o tamanho dos elementos visuais:

```

```fig:exemplo{caption="Figura redimensionada" width="300" height="200"}
imagem.jpg
```

```

- Valores numéricos são interpretados como pixels: `width="400"` → 400px
- Unidades CSS são suportadas: `width="50%"`, `width="10cm"`, `height="3in"`
- Se apenas um atributo for informado, o outro é ajustado proporcionalmente (mantendo aspect ratio)
- Para gráficos (`chart:`), os valores padrão são 600×400 pixels

- Ambos os atributos são opcionais

1.2.4.2 Atributo source

O atributo `source` define a fonte da ilustração conforme ABNT:

Tabela 1 – Sintaxe do atributo source

| Sintaxe | Resultado |
|----------------------------------|------------------------------------|
| <code>source="Autor"</code> | Fonte: Autor |
| <code>source="@silva2024"</code> | Fonte: (SILVA 2024) — via citeproc |
| (omitido) | Fonte: Elaborado pelo autor |
| Fonte: O autor | |

- **Texto literal:** Renderiza como “Fonte: {texto}”
 - **Citação BibTeX:** `source="@chave"` integra com citeproc, usando a formatação do arquivo CSL
 - **Omitido:** Aplica automaticamente “Fonte: Elaborado pelo autor” (padrão ABNT)
- Exemplo com citação bibliográfica:

Figura 4 – Capivara — exemplo de figura com fonte por citação



Fonte: IBGE (1993)

1.3 Tabelas, Quadros e Listagens

A ABNT distingue **tabela** de **quadro**. O SpecCompiler trata essa distinção como tipos diferentes — não como uma opção de formatação — e acrescenta um terceiro tipo, a **listagem**, para código e texto preformatado.

Cada um tem legenda, contador e lista pré-textual próprios.

1.3.1 A Diferença

Conforme as normas de apresentação tabular do Instituto Brasileiro de Geografia e Estatística (IBGE) (IBGE, 1993):

- **Tabela:** dados numéricos ou estatísticos. Bordas abertas nas laterais — apenas linhas horizontais no topo, sob o cabeçalho e na base (estilo de três linhas).
- **Quadro:** informações textuais. Bordas fechadas nos quatro lados e grade completa, com todas as células delimitadas.

A [Listagem 2](#) resume as duas formatações:

Listagem 2 – Diferença entre Tabela e Quadro conforme ABNT/IBGE

| TABELA | QUADRO |
|------------------------------|-----------------------------------|
| Dados numéricos/estatísticos | Informações textuais |
| Bordas abertas nas laterais | Bordas fechadas nos quatro lados |
| Só linhas horizontais | Grade completa (todas as células) |
| Legenda "Tabela N" | Legenda "Quadro N" |

Fonte: IBGE (1993)

1.3.2 Escolhendo o Tipo

Cada sintaxe tem um padrão que corresponde ao seu uso mais comum, e uma variante para o caso oposto. O [Quadro 1](#) resume o mapeamento:

Quadro 1 – Sintaxes tabulares e a formatação resultante

| Sintaxe | Resultado | Quando usar |
|--|-----------|---|
| <code>csv:</code> | Tabela | Dados numéricos em formato CSV |
| <code>csv-q:</code> | Quadro | Informações textuais em formato CSV |
| <code>list-table:</code> | Quadro | Informações textuais em múltiplas colunas |
| <code>list-table-t:</code> | Tabela | Dados numéricos em formato list-table |
| <code>listing: / src.<ext>:</code> | Listagem | Código ou texto preformatado, com moldura |

Fonte: Elaboração própria

O padrão segue a natureza de cada sintaxe: CSV nasceu para dados numéricos, e a sintaxe `list-table` existe justamente para células com texto longo. As variantes `csv-q:` e `list-table-t:` cobrem as exceções.

Os três tipos são numerados de forma independente: [Tabela 1](#), [Quadro 1](#) e [Listagem 1](#) podem coexistir. Cada um alimenta sua própria lista pré-textual — Lista de Tabelas, Lista de Quadros e Lista de Listagens.

1.3.3 Sintaxe CSV — Tabela

Para dados numéricos, a sintaxe CSV é a mais concisa. A [Tabela 2](#) renderiza como tabela, com as laterais abertas:

| Tabela 2 – Dados regionais — exemplo de tabela CSV | | | |
|--|------|------|------|
| Região | 2022 | 2023 | 2024 |
| Norte | 150 | 175 | 200 |
| Nordeste | 280 | 310 | 350 |
| Centro-Oeste | 120 | 140 | 160 |
| Sudeste | 450 | 520 | 580 |
| Sul | 200 | 230 | 260 |

Fonte: Dados fictícios

1.3.4 Sintaxe list-table — Quadro

A sintaxe `list-table` acomoda células com texto longo, e por isso renderiza como quadro. Cada linha começa com `*` - e cada célula seguinte com `-`. As linhas iniciadas por `>` configuram a tabela.

O [Quadro 2](#) demonstra a estrutura completa:

| Quadro 2 – Elementos pré-textuais conforme NBR 14724 | | |
|--|-------------|---|
| Elemento | Obrigatório | Descrição |
| Capa | Sim | Identificação do trabalho |
| Folha de rosto | Sim | Dados essenciais do trabalho |
| Ficha catalográfica | Não | Verso da folha de rosto |
| Errata | Não | Lista de correções |
| Folha de aprovação | Sim | Assinaturas da banca |
| Dedicatória | Não | Homenagem a pessoas |
| Agradecimentos | Não | Reconhecimento a contribuições |
| Epígrafe | Não | Citação relacionada ao tema |
| Resumo em português | Sim | Síntese do trabalho
(NBR 6028, 2003) |
| Resumo em língua estrangeira | Sim | Abstract |

| Elemento | Obrigatório | Descrição |
|-----------------------|-------------|---|
| Lista de ilustrações | Não | Índice de figuras |
| Lista de tabelas | Não | Índice de tabelas |
| Lista de abreviaturas | Não | Glossário de siglas |
| Sumário | Sim | Índice de conteúdo (NBR 6027, 2012) |

Fonte: NBR 14724 ([2011](#))

Os parâmetros de configuração aceitos são `header-rows` (quantas linhas iniciais formam o cabeçalho), `aligns` (alinhamento por coluna: `l`, `c` ou `r`) e `widths` (larguras relativas).

1.3.5 Listagens

Uma **listagem** é um bloco de texto preformatado com moldura, onde as quebras de linha têm significado — código-fonte, pseudocódigo, saída de terminal, diagramas em ASCII. Diferente do quadro, não tem células.

Há duas sintaxes:

1. `listing:label` — sem realce de sintaxe. Para texto estruturado, pseudocódigo ou blocos que não são de nenhuma linguagem em particular.
2. `src.<ext>:label` — com realce de sintaxe. O `<ext>` indica a linguagem (`lua`, `c`, `python`, `js`, `sql`...), colorizada via Pandoc Skyslighting.

Ambas aceitam `caption` e `source`, e recebem moldura no template ABNT.

1.3.5.1 Listagem Textual (sem realce)

A [Listagem 3](#) demonstra a sintaxe `listing:` para texto estruturado:

Listagem 3 – Principais vantagens do SpecCompiler

1. SINTAXE SIMPLES
Markdown é intuitivo e legível mesmo em formato texto puro.
Não requer conhecimento de LaTeX ou linguagens complexas.
2. CONFORMIDADE AUTOMÁTICA
O template ABNT cuida de margens, fontes, espaçamentos.
Validação estrutural impede documentos malformados.
3. EXTENSIBILIDADE
Sistema de templates permite criar novos formatos.
O template ABNT estende o template padrão (default).
4. INTEGRAÇÃO COM DADOS
Gráficos podem consultar o banco de dados.
Views SQL alimentam visualizações automaticamente.

Fonte: Elaboração própria

1.3.5.2 Listagem de Código (com realce)

A [Listagem 4](#) demonstra a sintaxe `src.c`: para código-fonte com realce de sintaxe:

Listagem 4 – Programa Hello World em C

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Fonte: Elaboração própria

Outras linguagens suportadas incluem `src.lua`:, `src.python`:, `src.java`:, `src.js`: e `src.sql`:. A lista completa depende do Pandoc Skylighting.

1.3.6 Variantes: Invertendo o Padrão

Quando o conteúdo não segue a natureza da sintaxe, use a variante. O [Quadro 3](#) usa CSV para informação textual, e por isso pede `csv-q`: para sair com grade completa:

Quadro 3 – Estilos de formatação em Markdown

| Sintaxe | Descrição |
|--------------------|---------------------------|
| **Negrito** | Ênfase forte |
| <i>*Itálico*</i> | Ênfase leve |
| `Código` | Elemento de código inline |
| [Link](url) | Hiperlink |

Fonte: Elaboração própria

Na direção oposta, dados numéricos às vezes precisam da sintaxe `list-table` por uma razão prática: **em português o separador decimal é a vírgula**, que também separa os campos do CSV. Escrito sem aspas, `3,0` vira duas colunas — e o valor excedente é descartado silenciosamente:

```
Elemento,Medida
Superior,3,0      → a célula vira "3"; o ",0" se perde
Superior,"3,0"    → correto, mas exige aspas em cada valor
```

A [Tabela 3](#) evita o conflito de vez, e `list-table-t`: garante a formatação de tabela:

Tabela 3 – Margens e recuos exigidos pela NBR 14724

| Elemento | Medida (cm) | Tolerância (cm) |
|-----------------|-------------|-----------------|
| Margem superior | 3,0 | 0,5 |
| Margem inferior | 2,0 | 0,5 |

| Elemento | Medida (cm) | Tolerância (cm) |
|--------------------|-------------|-----------------|
| Margem esquerda | 3,0 | 0,5 |
| Margem direita | 2,0 | 0,5 |
| Recuo de parágrafo | 1,25 | 0,25 |

Fonte: NBR 14724 (2011)

1.3.7 Equações e Fórmulas com AsciiMath

O SpecCompiler utiliza AsciiMath para expressões matemáticas — uma sintaxe mais simples e legível que LaTeX. A [Tabela 4](#) compara as duas notações:

Tabela 4 – Comparação entre AsciiMath e LaTeX

| Expressão | AsciiMath | LaTeX |
|--------------------|--------------------------------------|------------------------------------|
| Fração | x/y | $\frac{x}{y}$ |
| Raiz quadrada | $\text{sqrt}(x)$ | $\sqrt{x}$ |
| Potência | x^2 | x^2 |
| Subscrito | x_i | x_i |
| Somatório | $\text{sum}_{(i=1)}^n i$ | $\sum_{i=1}^n i$ |
| Integral | $\text{int}_0^1 f(x) dx$ | $\int_0^1 f(x) dx$ |
| Fórmula quadrática | $(-b \pm \text{sqrt}(b^2-4ac))/(2a)$ | $\frac{-b \pm \sqrt{b^2-4ac}}{2a}$ |

Fonte: Elaboração própria

1.3.7.1 Sintaxe Inline

Para inserir matemática no meio do texto, use a sintaxe `\math: expressão`. Exemplos:

- O teorema de Pitágoras: $a^2 + b^2 = c^2$
- Área do círculo: $A = \pi r^2$
- Somatório: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

1.3.7.2 Sintaxe em Bloco (Equação Numerada)

Para equações destacadas e numeradas, use:

$$a^2 + b^2 = c^2 \quad (1)$$

A [Equação 1](#) demonstra o teorema fundamental da geometria euclidiana.

A fórmula quadrática de Bhaskara ([Equação 2](#)) fornece as raízes de equações do segundo grau:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (2)$$

A integral de Gauss ([Equação 3](#)) é fundamental na teoria das probabilidades e estatística:

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} \quad (3)$$

A entropia de Shannon ([Equação 4](#)) quantifica a informação média em uma fonte de dados:

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i) \quad (4)$$

As equações demonstram a capacidade do SpecCompiler de renderizar fórmulas matemáticas complexas usando notação AsciiMath, que é mais intuitiva que LaTeX para a maioria dos casos.

1.3.7.3 Referência Rápida AsciiMath

- Operações básicas: +, -, *, /, =, !=, <, >, <=, >=
- Símbolos gregos: alpha, beta, gamma, delta, pi, theta, omega
- Funções: sin, cos, tan, log, ln, exp, sqrt
- Agrupamento: parênteses (), colchetes [], chaves {}
- Matrizes: [[a,b],[c,d]] para matriz 2x2

1.4 Citações e Referências Bibliográficas

O SpecCompiler utiliza Citeproc para processar citações—não o tradicional toolchain BibTeX/Biber do LaTeX. Essa escolha simplifica significativamente o fluxo de trabalho: um único arquivo .bib e um estilo CSL são suficientes, sem necessidade de múltiplas compilações ou arquivos auxiliares (.aux, .bbl, .blg, .bcf, .run.xml). O processamento segue as normas NBR 10520 ([NBR 10520, 2002](#)) e NBR 6023 ([NBR 6023, 2018](#)).

1.4.1 Tipos de Citação

Citação indireta (entre parênteses): A referência aparece ao final da frase.

A formatação de trabalhos acadêmicos deve seguir normas padronizadas ([NBR 14724, 2011](#)).

Citação direta (no texto): O autor faz parte da sentença.

Segundo NBR 14724 ([2011](#)), os trabalhos acadêmicos devem conter elementos pré-textuais, textuais e pós-textuais.

1.4.2 Múltiplas Citações

Para citar múltiplas obras simultaneamente, separe as chaves com ponto e vírgula:

Diversos autores tratam da normalização de documentos acadêmicos ([NBR 10520, 2002](#); [NBR 14724, 2011](#); [NBR 6023, 2018](#)).

1.4.3 Configuração: Citeproc e CSL

O processamento de citações utiliza dois componentes:

1. **Citeproc**: Processador de citações integrado ao Pandoc que converte `[chave](@cite)` em citações formatadas
2. **CSL (Citation Style Language)**: Arquivo XML que define o estilo de formatação das citações

O template ABNT já fornece o estilo de citação (`abnt.csl`, declarado em `models/abnt/config.lua`); o `project.yaml` precisa apenas indicar a bibliografia:

```
# Configuração de bibliografia (citeproc)
bibliography: references.bib
# csl: opcional – só declare aqui para sobrescrever o estilo do modelo
```

O arquivo CSL determina:

- Formato da citação no texto: (Autor, ano) vs [Autor, ano]
- Formato das entradas na bibliografia
- Ordenação das referências
- Regras de abreviação (et al.)

O estilo ABNT segue a NBR 10520 ([NBR 10520, 2002](#)) para citações e NBR 6023 ([NBR 6023, 2018](#)) para referências.

1.5 Referências Cruzadas

O SpecCompiler suporta referências cruzadas para figuras, tabelas, quadros, equações e seções. O texto exibido (“Figura 1”, “Quadro 3”) é resolvido na compilação — nunca digitado à mão:

- Figuras: conforme demonstrado na [Figura 1](#)
- Tabelas: os dados da [Tabela 2](#) mostram a evolução regional
- Quadros: o [Quadro 2](#) enumera os elementos
- Listagens: a [Listagem 2](#) contrasta as duas formatações
- Equações: a [Equação 2](#) resolve o segundo grau

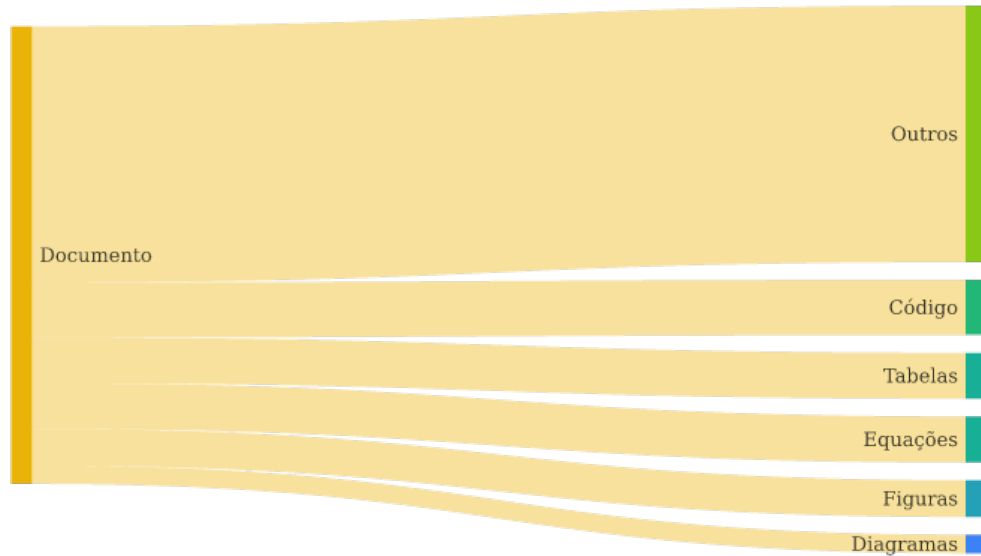
A sintaxe usa `[prefixo:identificador](#)`, onde o prefixo é o tipo do bloco referenciado:

| | |
|---------------------------------|--------------|
| <code>[fig:id](#)</code> | → Figura N |
| <code>[csv:id](#)</code> | → Tabela N |
| <code>[list-table:id](#)</code> | → Quadro N |
| <code>[listing:id](#)</code> | → Listagem N |
| <code>[math:id](#)</code> | → Equação N |
| <code>[chart:id](#)</code> | → Gráfico N |

Qualquer alias do tipo funciona como prefixo: `[tab:id](#)` e `[csv:id](#)` apontam para o mesmo tipo Tabela.

O sistema infere o **tipo de relação** (`XREF_FIGURE`, `XREF_TABLE`, `XREF_QUADRO`, `XREF_LISTING`, `XREF_MATH`) para cada elemento flutuante durante a compilação. Referências não resolvidas interrompem o build, o que impede legendas órfãs no documento final. O gráfico a seguir ([Gráfico 3](#)) mostra a distribuição desses tipos por seção—dados extraídos diretamente do banco de dados do documento:

Gráfico 3 – Tipos XREF por Seção — inferência de relações em tempo de compilação
Seções → Tipos de Referência



Fonte: O autor

1.6 Siglas e Abreviaturas

O SpecCompiler gerencia siglas automaticamente. Na primeira ocorrência, o termo completo é exibido com a sigla entre parênteses. Nas ocorrências seguintes, apenas a sigla aparece.

Para definir uma sigla, use:

```
`sigla: Termo Completo (SIGLA)`
```

Ao longo deste documento, as siglas ABNT, IBGE, UML, IEEE e APA foram definidas dessa forma. Todas aparecem automaticamente na Lista de Abreviaturas e Siglas, em ordem alfabética, sem qualquer marcação manual.

A definição pode aparecer em qualquer capítulo — a primeira ocorrência no documento montado é que determina onde o termo por extenso é exibido.

1.7 Peculiaridades das Normas ABNT

As normas ABNT para trabalhos acadêmicos incluem requisitos complexos que o SpecCompiler implementa automaticamente.

1.7.1 Numeração de Páginas

A NBR 14724 ([NBR 14724, 2011](#)) estabelece que os elementos pré-textuais são **contados, mas não numerados**. A numeração visível começa apenas na parte textual, em algarismos arábicos.

Curiosamente, o uso de algarismos romanos nos pré-textuais (i, ii, iii...) **não é exigido pela norma** — é uma adição de algumas instituições ([Equipe abnTeX2, 2015](#)). O SpecCompiler suporta ambas as configurações.

1.7.2 Capítulos em Páginas Ímpares

Para impressão frente e verso, a norma exige que capítulos iniciem em páginas ímpares (lado direito do documento aberto). O SpecCompiler insere automaticamente páginas em branco quando necessário.

1.7.3 Margens Espelhadas

Para encadernação, as margens são espelhadas:

- Margem interna (encadernação): 3 cm
- Margem externa: 2 cm

1.7.4 Formatação de Tabelas e Quadros (Padrão IBGE)

Tabelas seguem o padrão do IBGE ([IBGE, 1993](#)):

- Bordas abertas nas laterais
- Apenas linhas horizontais (topo, separador, base)
- Título acima da tabela
- Fonte abaixo

Quadros, por conterem informação textual, recebem grade completa e bordas fechadas.

O SpecCompiler aplica cada geometria automaticamente a partir do tipo do bloco — ver o capítulo *Tabelas e Quadros*.

1.7.5 Template Extensível

O template ABNT é apenas um exemplo das capacidades do SpecCompiler. O sistema de extensões permite criar templates para qualquer padrão documental — Institute of Electrical and Electronics Engineers (IEEE), American Psychological Association (APA), normas corporativas, etc.

O template ABNT estende o template `default`, herdando funcionalidades como gráficos, PlantUML e tabelas (`list-table`, `CSV`, `TSV`), e adicionando tipos específicos para a estrutura acadêmica brasileira — entre eles o quadro tabular, ausente em normas de outros países.

1.8 Conclusão

Este documento demonstrou como utilizar o template ABNT do SpecCompiler para criar trabalhos acadêmicos em conformidade com a NBR 14724 (NBR 14724, 2011). Através de exemplos práticos, foram apresentados:

1. Inserção de figuras com legendas padronizadas (Figura 1)
2. Gráficos gerados por funções Lua (Gráfico 1)
3. Diagramas PlantUML (Figura 3)
4. Quadros com sintaxe list-table (Quadro 2)
5. Tabelas com sintaxe CSV (Tabela 2)
6. Listagens emolduradas para texto e código (Listagem 3)
7. Citações bibliográficas conforme ABNT
8. Referências cruzadas automáticas
9. Gerenciamento de siglas

1.8.1 Uma Mudança de Paradigma

Mais do que uma ferramenta de formatação, o SpecCompiler representa uma mudança de paradigma na produção de documentos acadêmicos brasileiros. Ao eliminar o LaTeX do pipeline de geração, rompe-se com décadas de dependência de um ecossistema notoriamente complexo.

A própria nomenclatura do ecossistema TeX revela sua fragmentação: TeX é o sistema de tipografia original de Donald Knuth (1978); LaTeX é uma camada de macros sobre o TeX criada por Leslie Lamport; pdfTeX, XeTeX e LuaTeX são diferentes *engines* que processam o código-fonte; e distribuições como TeX Live e MiKTeX empacotam milhares de pacotes com dependências cruzadas. Para o usuário iniciante, a distinção entre esses componentes raramente é clara—e quando algo falha, a depuração exige conhecimento de múltiplas camadas.

Plataformas como Overleaf (Digital Science, 2024) mitigaram parte dessa complexidade ao oferecer um ambiente web pré-configurado, democratizando o acesso ao LaTeX. Porém, ao custo de *lock-in*: o documento existe apenas na nuvem do fornecedor, e a colaboração depende de todos os participantes terem conta na plataforma.

O formato DOCX de saída do SpecCompiler não é uma limitação, mas uma característica estratégica. Diferentemente do PDF—um formato de visualização, não de edição—o DOCX permite colaboração nativa: controle de alterações, comentários em linha, e revisão por orientadores e bancas usando ferramentas já instaladas em seus computadores (Microsoft Word, LibreOffice, Google Docs). Não há necessidade de instalar software especializado ou criar contas em plataformas proprietárias.

O SpecCompiler automatiza a formatação e validação do documento, permitindo que o autor concentre-se no conteúdo.

1.8.2 Documentação e Relatórios: A Linha que Desaparece

Ferramentas tradicionais separam **documentação** (conteúdo estático, escrito manualmente) de **relatórios** (conteúdo dinâmico, gerado de dados). O SpecCompiler dissolve essa fronteira.

Neste documento, elementos como:

- a seção `## Sumário` — sumário gerado automaticamente da estrutura
 - a seção `## Lista de Figuras` — lista de figuras extraída do banco de dados
 - as seções `## Lista de Quadros` e `## Lista de Listagens` — geradas dos contadores
 - a seção `## Lista de Abreviaturas e Siglas` — siglas populadas durante a compilação
 - `chart:xref-usage{query="..."}` — gráfico que consulta views SQL
- ...não são conteúdo estático. São **consultas materializadas** que se atualizam a cada

build. O documento é simultaneamente **documentação** (texto autoral) e **relatório** (dados do sistema de tipos).

Essa fusão é possível porque o SpecCompiler armazena tudo em SQLite: objetos, relações, atributos, hierarquias. Views SQL podem agregar, filtrar e projetar esses dados—e gráficos ECharts podem visualizá-los. O template apenas define **quais views existem e como renderizá-las**.

O mesmo mecanismo que gera “Lista de Figuras” numa monografia pode gerar “Matriz de Rastreabilidade” num SRS ou “Relatório de Cobertura” num TRR. A diferença é o template e as views—não o motor.

1.9 Trabalhos Futuros

O desenvolvimento do SpecCompiler segue duas direções prioritárias:

1.9.1 Editor Colaborativo em Tempo Real

Integração com um editor web que permita múltiplos autores editarem o mesmo documento simultaneamente, similar à experiência do Google Docs ou Overleaf—mas com o documento-fonte em Markdown e armazenamento local ou em repositórios Git, evitando dependência de plataformas proprietárias.

1.9.2 Interoperabilidade DOCX (Round-Trip)

Atualmente, o fluxo é unidirecional: Markdown $\rightarrow$ DOCX. Um objetivo futuro é suportar importação de arquivos DOCX existentes, convertendo-os para a estrutura Markdown do SpecCompiler. Isso permitiria:

- Migrar trabalhos iniciados em Word para o ecossistema SpecCompiler
- Incorporar revisões feitas diretamente no DOCX de volta ao fonte Markdown
- Ciclos iterativos de edição (*round-trip*) sem perda de formatação semântica

Essa interoperabilidade bidirecional eliminaria a principal barreira de adoção: a necessidade de começar do zero em uma nova ferramenta.

REFERENCIAS

DIGITAL SCIENCE. **Overleaf: Online LaTeX Editor**. Disponível em: <<https://www.overleaf.com>>. Acesso em: 15 dez. 2024.

EQUIPE ABNTEX2. **HowTo: Numeração romana nos pré-textuais**. Disponível em: <<https://github.com/abntex/abntex2/wiki/HowToPretextuaisRomanos>>. Acesso em: 15 dez. 2024.

IBGE. **Normas de apresentação tabular**. 3. ed. Rio de Janeiro: Centro de Documentação e Disseminação de Informações, 1993.

NBR 10520: Informação e documentação — Apresentação de citações em documentos. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2002. p. 7

NBR 14724: Informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2011. p. 15

NBR 14724: Informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2024. p. 12

NBR 6023: Informação e documentação — Referências — Elaboração. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2018. p. 74

NBR 6024: Informação e documentação — Numeração progressiva das seções de um documento. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2012. p. 4

NBR 6027: Informação e documentação — Sumário — Apresentação. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2012. p. 3

NBR 6028: Informação e documentação — Resumo — Apresentação. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2003. p. 2

POSIT PBC. **Quarto: An open-source scientific and technical publishing system.**, 2022. Disponível em: <<https://quarto.org>>

VIEIRA, Eduardo de Santana Medeiros. **Limarka: Markdown para trabalhos acadêmicos com ABNT.**, 2016. Disponível em: <<https://github.com/abntex/limarka>>

APÊNDICE A – Referência Rápida de Sintaxe

A.1 Atributos do Documento

```
> institution: Nome da Instituição
> author: Nome do Autor
> title: Título do Trabalho
```

A.2 Figuras

```
```fig:id{caption="Legenda" source="Fonte" width="400" height="300"}
caminho/imagem.jpg
```
```

A.3 Diagramas PlantUML

```
```puml:id{caption="Título do diagrama" width="600"}
@startuml
...
@enduml
```
```

A.4 Gráficos

```
```chart:id{query="nome_da_view" caption="Título" width="800" height="500"}
{ ...configuração ECharts... }
```

```chart:id{generator="gauss" mean="0" sigma="1" caption="Título" height="5cm"}
{ ...configuração ECharts... }
```
```

- `query`: consulta uma view SQL sobre o banco do documento
- `generator` (ou `view`): invoca um módulo Lua de `types/views/`

A.5 Atributos de Dimensões e Fonte

Aplicáveis a `fig:`, `puml:`, `chart:`

- `width`: Largura (px, %, cm, in)
- `height`: Altura (px, %, cm, in)
- `source`: Fonte — texto, `@citação`, ou omitido → “Elaborado pelo autor”

A.6 Tabelas (dados numéricos)

```
```csv:id{caption="Título" source="Fonte"}
Col1,Col2,Col3
Val1,Val2,Val3
```

```list-table-t:id{caption="Título" source="Fonte"}
> header-rows: 1
> aligns: l,r
* - Coluna
 - Valor
* - Norte
 - 150
```
```

Renderiza com bordas abertas nas laterais (estilo IBGE) e legenda Tabela N.

A.7 Quadros (informação textual)

```

```list-table:id{caption="Título" source="Fonte"}
> header-rows: 1
> aligns: l,c,l
* - Col1
 - Col2
 - Col3
* - Valor1
 - Valor2
 - Valor3
```

```csv-q:id{caption="Título" source="Fonte"}
Termo,Definição
Tabela,Dados numéricos
```

```

Renderiza com grade completa e legenda Quadro N.

Parâmetros de `list-table` (linhas iniciadas por `>`):

- `header-rows`: número de linhas de cabeçalho
- `aligns`: alinhamento por coluna — l, c, r
- `widths`: larguras relativas das colunas

A.8 Listagens (código e texto preformatado)

```

```listing:id{caption="Título" source="Fonte"}
Conteúdo textual, sem realce de sintaxe
```

```src.c:id{caption="Título" source="Fonte"}
int main(void) { return 0; }
```

```

Renderiza com moldura e legenda Listagem N.

A.9 Equações

```

`math: a^2 + b^2 = c^2`           - inline

```math:id{caption="Título"}
x = (-b +- sqrt(b^2 - 4ac)) / (2a)
```

```

A.10 Citações

```

[chave] (@cite)                  - Citação entre parênteses
[chave] (@citep)                 - Citação no texto
[chave1;chave2] (@cite)          - Múltiplas citações

```

A.11 Notas de Rodapé

```

Texto com nota[^id].           - Referência inline

[^id]: Conteúdo da nota de rodapé.

```

A.12 Referências Cruzadas

```

[fig:id] (#)                    → Figura N
[csv:id] (#)                    → Tabela N
[list-table:id] (#)             → Quadro N
[listing:id] (#)                → Listagem N
[math:id] (#)                   → Equação N

```

```
[chart:id] (#)      → Gráfico N
[puml:id] (#)       → Figura N
```

O prefixo pode ser qualquer alias do tipo. Referências não resolvidas interrompem o build.

A.13 Siglas

```
`sigla: Termo Completo (SIGLA)`
```

A.14 Inclusão de Arquivos

Use `include` para compor um documento a partir de vários arquivos Markdown. Os caminhos são relativos ao arquivo que contém a diretiva:

```
```include
chapters/introducao.md
chapters/figuras-ilustracoes.md
chapters/referencias.md
chapters/appendice-sintaxe.md
```
```

O conteúdo incluído participa da mesma estrutura lógica do documento: cabeçalhos, figuras, tabelas, citações, siglas e referências cruzadas são processados em conjunto.

A.15 Configuração (project.yaml)

```
project:
  code: MONOGRAFIA_EXAMPLE
  name: Exemplo de Monografia ABNT

template: abnt
style: academico
output_dir: build/

doc_files:
  - monografia.md

# Bibliografia (citeproc); o estilo CSL é fornecido pelo modelo ABNT.
bibliography: references.bib

outputs:
  - format: docx
    path: "{spec_id}.docx"

docx:
  update_fields: true    # atualiza sumário e listas ao gerar
  export_pdf: true      # exporta PDF via LibreOffice
```

A.16 Seções Pré-textuais Automáticas

Basta declarar o cabeçalho; o conteúdo é gerado na compilação:

```
## Sumário
## Lista de Figuras
## Lista de Tabelas
## Lista de Quadros
## Lista de Listagens
## Lista de Abreviaturas e Siglas
```

ANEXO A – Normas ABNT Consultadas

Lista das normas da ABNT consultadas para desenvolvimento do template ([NBR 10520, 2002](#); [NBR 14724, 2011, 2024](#); [NBR 6023, 2018](#); [NBR 6024, 2012](#); [NBR 6027, 2012](#); [NBR 6028, 2003](#)):

- **NBR 14724:2011** — Trabalhos acadêmicos — Apresentação
- **NBR 14724:2024** — Trabalhos acadêmicos — Apresentação (atualização)
- **NBR 6023:2018** — Referências — Elaboração
- **NBR 6028:2003** — Resumo — Apresentação
- **NBR 6027:2012** — Sumário — Apresentação
- **NBR 6024:2012** — Numeração progressiva das seções
- **NBR 10520:2002** — Citações em documentos